

DevSync — Phase 1 Implementation Prompt

You are helping me build **DevSync**, a production-style MERN stack developer collaboration and project management platform.

I am a **fresher learning MERN**, so do not treat this as a simple code-generation task. I want to understand the architecture and the reason behind every important implementation decision.

We are currently implementing **ONLY PHASE 1: PROJECT FOUNDATION & INITIAL ARCHITECTURE**.

Do not implement features belonging to later phases unless they are required for the Phase 1 foundation.

PROJECT CONTEXT

DevSync will eventually provide:

- User authentication
- Organizations
- Role-based access control
- Projects
- Tasks
- Kanban boards
- Comments
- Team chat
- Real-time collaboration
- Notifications
- Activity tracking
- AI-powered issue analysis
- Analytics
- Redis caching
- Security
- Testing
- Docker
- CI/CD
- Deployment

However, these features belong to later phases.

For Phase 1, we are building only the foundation.

PHASE 1 GOAL

At the end of Phase 1, the following architecture must work:

React Frontend ↓ Axios ↓ Express REST API ↓ Node.js ↓ MongoDB

The frontend must be able to call a backend health-check API and display the result.

The backend must successfully connect to MongoDB.

TECHNOLOGY REQUIREMENTS

Frontend

Use:

- React
- Vite
- JavaScript
- React Router
- Axios

Do not introduce TypeScript unless explicitly requested.

Backend

Use:

- Node.js
- Express.js
- JavaScript
- Mongoose
- dotenv
- cors
- helmet

Use ES Modules.

Database

Use:

- MongoDB
 - Mongoose
-

REQUIRED PROJECT STRUCTURE

Create a monorepo approximately like this:

devsync/

```
client/

  public/

  src/
    assets/
    components/
    layouts/
    pages/
    routes/
    services/
    store/
    hooks/
    utils/

    App.jsx
    main.jsx

  .env.example
  package.json
  vite.config.js

server/

  config/
    database.js
    env.js

  controllers/

  middleware/
    error.middleware.js
    notFound.middleware.js

  models/

  routes/
    health.routes.js

  services/

  utils/

  app.js
  server.js

  .env.example
  package.json

.gitignore
```

```
README.md
package.json
```

The exact structure can be adjusted if there is a strong architectural reason, but do not unnecessarily over-engineer it.

IMPLEMENTATION RULES

Rule 1 — Teach before implementing

Before giving me a significant piece of code, briefly explain:

1. What we are building.
2. Why we need it.
3. Where it belongs in the architecture.
4. How it will interact with the other components.

Keep explanations beginner-friendly.

Rule 2 — Build incrementally

Do NOT give me the entire project as one giant code dump.

Build Phase 1 in small milestones.

For each milestone:

1. Explain the concept.
 2. Show the commands.
 3. Show the files we need to create/change.
 4. Provide the code.
 5. Explain important parts of the code.
 6. Tell me how to run it.
 7. Tell me what output I should expect.
 8. Give me a small verification checklist.
 9. Only then move to the next milestone.
-

MILESTONE 1 — PROJECT INITIALIZATION

Set up the root project.

Create:

devsync/

```
client/  
  
server/  
  
README.md  
  
.gitignore  
  
package.json
```

Explain why we are using a monorepo-style structure.

Do not add unnecessary packages.

MILESTONE 2 — FRONTEND SETUP

Create the React application using Vite.

Configure:

- React
- React Router
- Axios

Create a basic DevSync application shell.

The initial UI can simply contain:

DevSync

Developer Collaboration Platform

Backend Status: Checking...

Do not build the complete dashboard yet.

MILESTONE 3 — BACKEND SETUP

Create the Express server.

Use a separation similar to:

server.js ↓ app.js ↓ middleware + routes

The server should:

- Load environment variables.
 - Initialize Express.
 - Configure JSON parsing.
 - Configure CORS.
 - Configure Helmet.
 - Register routes.
 - Register 404 middleware.
 - Register error middleware.
 - Start listening on the configured port.
-

MILESTONE 4 — ENVIRONMENT CONFIGURATION

Create:

server/.env.example client/.env.example

The backend environment should support:

PORT MONGODB_URI CLIENT_URL NODE_ENV

Do not commit actual secrets.

Explain:

- What `.env` is.
 - Why secrets should not be committed.
 - Difference between `.env` and `.env.example`.
 - Why environment-specific configuration is useful.
-

MILESTONE 5 — MONGODB CONNECTION

Implement:

server/config/database.js

Use Mongoose.

Create a reusable database connection function.

The server should establish the MongoDB connection before starting the application server, or otherwise handle startup failure safely.

Explain:

- What MongoDB is.
- What Mongoose does.
- What a connection URI is.
- Why database logic should be separated from `server.js`.
- What happens if MongoDB is unavailable.

Expected console output should clearly indicate successful database connection.

MILESTONE 6 — HEALTH CHECK API

Create:

GET /api/health

The API should return something similar to:

```
{ "success": true, "message": "DevSync API is running" }
```

Structure the route properly instead of putting the endpoint directly inside `server.js`.

Explain:

Route → Controller → Response

Even if the health endpoint is simple, use an architecture that can scale to future modules.

MILESTONE 7 — ERROR HANDLING

Implement centralized error handling.

Create:

middleware/error.middleware.js

middleware/notFound.middleware.js

Unknown routes should return an appropriate 404 response.

Errors should return a consistent structure such as:

```
{ "success": false, "message": "Route not found" }
```

Explain:

- Why centralized error handling matters.
- Difference between 404 and 500.
- Why internal stack traces should not be exposed in production.

MILESTONE 8 — FRONTEND API SERVICE

Create a centralized Axios instance.

For example:

`src/services/api.js`

Configure the API base URL through environment variables.

Do NOT hardcode:

`http://localhost:5000`

throughout the application.

Explain why centralized API configuration becomes important when we later add:

- Authentication
- Projects
- Tasks
- Notifications
- AI APIs

MILESTONE 9 — FRONTEND ↔ BACKEND CONNECTION

The React frontend should call:

`GET /api/health`

and display the result.

Example:

DevSync

Backend Status: ✓ Connected

The UI should handle at least:

Loading Success Failure

For example:

Backend Status: Checking...

Backend Status: Connected

Backend Status: Unable to connect

Explain the complete request lifecycle:

React → Axios → HTTP → Express → Route → Controller → Response → Axios → React state → UI

MILESTONE 10 — GIT/GITHUB FOUNDATION

Initialize Git.

Create an appropriate `.gitignore`.

Make sure these are NOT committed:

`.env` `node_modules` `build/dist` `output` `logs` OS-specific files

Create an initial commit.

Recommended branch structure:

`main` `development`

Feature branches can later follow:

`feature/<feature-name>`

Explain:

- Git
- GitHub
- branch
- commit
- push
- pull request

Do not assume I already understand these concepts.

MILESTONE 11 — README

Create a professional README containing:

DevSync

Short project description.

Current Phase

Phase 1 — Project Foundation

Tech Stack

Frontend: React, Vite, Axios

Backend: Node.js, Express

Database: MongoDB, Mongoose

Project Structure

Show the important folder structure.

Local Setup

Explain:

1. Clone repository.
2. Install dependencies.
3. Configure environment variables.
4. Start MongoDB.
5. Start backend.
6. Start frontend.

API

Document:

GET /api/health

Development Status

Phase 1: Foundation — Completed/In Progress

Do not claim features that haven't been implemented.

CODE QUALITY REQUIREMENTS

Follow these principles:

- Keep files focused on one responsibility.
- Avoid unnecessary abstraction.
- Use meaningful variable names.
- Use `async/await`.
- Use consistent error handling.
- Avoid duplicate code.
- Keep configuration separate from business logic.
- Do not hardcode secrets.
- Do not hardcode API URLs.
- Do not put database logic directly into routes.
- Do not put everything into `server.js`.
- Do not over-engineer Phase 1.

SECURITY FOUNDATION

Implement only the security relevant to Phase 1:

- Helmet
- CORS configuration
- Environment variables
- Safe error responses
- No hardcoded secrets

Do NOT implement JWT authentication yet.

Authentication belongs to Phase 2.

Do NOT add Redis yet.

Redis belongs to the performance phase.

Do NOT add Socket.IO yet.

Real-time communication belongs to a later phase.

Do NOT add AI yet.

AI belongs to the AI phase.

IMPORTANT LEARNING REQUIREMENT

Whenever we introduce a technology, explain it simply.

For example:

If introducing Axios:

Explain:

"What problem does Axios solve?"

If introducing Express:

Explain:

"What does Express do on top of Node.js?"

If introducing Mongoose:

Explain:

"Why don't we directly write MongoDB operations everywhere?"

If introducing middleware:

Explain:

"What is middleware and when does it run?"

If introducing CORS:

Explain:

"Why is the browser blocking or allowing this request?"

PHASE 1 VERIFICATION

At the end, verify the following.

Frontend

- React application starts.
- React page loads.
- React can call backend.
- Loading state works.

- Success state works.
- Failure state works.

Backend

- Express starts.
- Environment variables load.
- Health API works.
- CORS works.
- Helmet is configured.
- 404 handling works.
- Error handling works.

Database

- MongoDB connection succeeds.
- Connection failure is handled.

Git

- `.env` is ignored.
- `node_modules` is ignored.
- Repository has meaningful commits.
- README exists.

FINAL SUCCESS TEST

The following must work:

1. Start MongoDB.
2. Start the DevSync backend.
3. Backend connects to MongoDB.
4. Start React frontend.
5. Open DevSync in the browser.
6. React calls `/api/health`.
7. Express receives the request.
8. Express returns the health response.
9. React displays "Backend Connected".
10. No secrets are exposed.
11. Project structure is ready for Phase 2.

AFTER PHASE 1

Do not automatically start Phase 2.

First provide:

1. Phase 1 completion summary.
2. Final folder structure.
3. Technologies used.
4. APIs created.
5. Important concepts learned.
6. Verification checklist.
7. Any technical debt or improvements.
8. Then wait for my instruction to begin Phase 2.

The goal is not merely to make the application run.

The goal is for me to understand **how a professional MERN application is structured and why each architectural decision was made.**